

Privacy-Preserving Fraud Detection using Federated Learning with Differential Privacy

A Project Report Submitted in Partial Fulfilment of the
Requirements for the Degree of Bachelor of Engineering
in Computer Science and Engineering (AI/ML)

Department of Computer Science and Engineering (AI/ML)
Ramaiah Institute of Technology, Bengaluru

Abstract

Centralized machine learning for financial fraud detection requires pooling transaction data from multiple institutions into a single repository, an approach that conflicts with data privacy regulations and inter-institutional data-sharing restrictions. This project implements a federated learning (FL) framework in which a fraud detection model is trained collaboratively across five simulated data-holding clients without any raw transaction data leaving its source. Using the Flower framework, the system implements and empirically compares two aggregation strategies, Federated Averaging (FedAvg) and FedProx, under a non-independent and identically distributed (non-IID) client data partition generated via a Dirichlet distribution, reflecting realistic institution-level data heterogeneity. Differential privacy is incorporated into local client training using Opacus, and the resulting privacy-utility tradeoff is measured with correctly composed cumulative epsilon accounting across communication rounds. The project further validates its privacy claims empirically through a loss-threshold membership inference attack, and investigates adaptive per-client noise scaling as a mitigation for non-uniform client dataset sizes. The system is evaluated against a centralized baseline model on accuracy, precision, recall, F1-score, and AUC. Results show that FedAvg ($F1 = 0.798$) matches or exceeds the centralized baseline ($F1 = 0.70$), that plain differential privacy composition under this setup produces an inherent conflict between meaningful privacy budgets and usable utility, and that adaptive per-client noise scaling based on dataset size alone can protect small clients but cannot simultaneously protect large, high-step-count clients, motivating per-client privacy calibration as a direction for future work.

Keywords: Federated Learning, FedAvg, FedProx, Differential Privacy, Membership Inference, Fraud Detection, Non-IID Data

Table of Contents

Abstract

Chapter 1: Introduction

Chapter 2: Literature Survey

Chapter 3: System Analysis

Chapter 4: System Design and Methodology

Chapter 5: Implementation

Chapter 6: Results and Discussion

Chapter 7: Conclusion and Future Scope

References

Chapter 1: Introduction

1.1 Background

Financial fraud detection models improve with access to larger and more diverse transaction datasets. In practice, however, transaction data is distributed across many independent financial institutions, and data protection regulations, competitive concerns, and contractual restrictions prevent this data from being pooled into a single centralized repository for model training. Federated Learning (FL) is a distributed machine learning paradigm that addresses this constraint: instead of transferring raw data to a central server, each participating institution (referred to as a client) trains a model locally on its own data, and only model parameter updates, not the underlying data, are shared with a central aggregating server.

1.2 Problem Statement

A fraud detection model trained on a single institution's data alone observes only a narrow slice of fraud patterns and typically underperforms a model trained on pooled data from multiple institutions. The central challenge addressed by this project is: how can multiple financial institutions collaboratively train a single, high-performing fraud detection model without ever sharing their raw transaction data, while also providing a formal, quantifiable guarantee that individual transactions cannot be reconstructed or inferred from the shared model updates?

1.3 Objectives

- Simulate a realistic federated learning environment with multiple institutions holding non-identically distributed (non-IID) fraud data.
- Implement and compare Federated Averaging (FedAvg) and FedProx as aggregation strategies under this non-IID setting.
- Integrate differential privacy into local client training and correctly measure the cumulative privacy budget (epsilon) across communication rounds.
- Empirically validate privacy claims using a membership inference attack.
- Investigate adaptive per-client noise calibration as a strategy to address heterogeneous client dataset sizes.
- Benchmark all federated variants against a centralized baseline model using standard classification metrics.

1.4 Scope

The project uses a publicly available, anonymized credit card transaction dataset and simulates federated clients by partitioning this dataset rather than sourcing data from real financial institutions. The federated learning environment is simulated on a single machine using the Flower framework's simulation engine, which emulates multiple independent clients through Ray's actor-based parallelism. The project's scope is restricted to a binary classification task (fraudulent vs. non-fraudulent transaction) using a lightweight feedforward neural network, since the project's emphasis is on the federated learning and privacy mechanisms rather than model architecture complexity.

Chapter 2: Literature Survey

2.1 Federated Learning and FedAvg

Federated Learning was introduced by McMahan et al. (2017) in the context of mobile keyboard prediction, proposing the Federated Averaging (FedAvg) algorithm. In FedAvg, a central server distributes a global model to a subset of clients, each client performs several epochs of local stochastic gradient descent on its own data, and the server aggregates the resulting model weights via a weighted average, typically weighted by the number of local training samples. This process repeats over multiple communication rounds until the global model converges. FedAvg's simplicity has made it the de facto baseline for federated learning research and deployment.

2.2 Non-IID Data and FedProx

A key assumption in early FL work was that client data is independently and identically distributed (IID); in practice, this rarely holds, since institutions differ systematically in the populations and behaviours their data captures. Li et al. (2020) proposed FedProx, which modifies FedAvg by adding a proximal regularization term to each client's local objective, penalizing local model updates that drift significantly from the global model. This is intended to stabilize training under statistical heterogeneity (non-IID data) and partial client participation.

2.3 Differential Privacy and DP-SGD

Differential Privacy (DP), formalized by Dwork et al., provides a mathematical guarantee that the presence or absence of any single individual's data in a dataset has a bounded effect on the output of an analysis, quantified by a privacy budget (epsilon). Abadi et al. (2016) introduced DP-SGD, which achieves this guarantee for neural network training by clipping per-sample gradients to a fixed norm and adding calibrated Gaussian noise before the optimizer step. The composition of privacy loss across many training steps is tracked using a privacy accountant; the Renyi Differential Privacy (RDP) accountant and the more recently developed Privacy loss Random Variable (PRV) accountant are two common approaches, differing in computational cost and the tightness of the resulting epsilon bound.

2.4 Differentially Private Federated Learning

Combining DP with FL introduces additional composition considerations: privacy loss accumulates both across local training steps within a client and across communication rounds. Prior work has noted that client heterogeneity in dataset size complicates uniform privacy guarantees, since a fixed noise multiplier applied identically across all clients results in disproportionately weaker protection for the largest client's many-sample training process and disproportionately strong noise injection relative to signal for the smallest client's few-sample training process. This tension is directly investigated in this project through the design of an adaptive per-client noise scaling mechanism.

2.5 Membership Inference Attacks

Membership Inference Attacks (MIA), studied by Shokri et al. and others, attempt to determine whether a specific data record was used to train a given model, typically by exploiting the observation that models tend to exhibit lower loss or higher confidence on training samples than on unseen samples, due to partial memorization. MIA success rates are commonly used as an empirical, practical measure of a model's privacy leakage, complementing the theoretical

epsilon guarantee provided by differential privacy.

Note: as with any AI-assisted literature summary, the citations above should be independently verified against the original publications before being included in a formal submission, as exact publication details cannot be confirmed without database access.

Chapter 3: System Analysis

3.1 Dataset

The project uses the publicly available Credit Card Fraud Detection dataset (Kaggle: [mlg-ulb/creditcardfraud](#)), containing 284,807 transactions made by European cardholders, of which 492 (0.173%) are labelled fraudulent. Features V1 through V28 are the result of a PCA transformation applied by the dataset's original authors to protect sensitive information; the Time and Amount features are provided in their original form. The extreme class imbalance in this dataset (fraud cases representing less than 0.2% of transactions) is a central analytical consideration throughout the project.

3.2 Non-IID Client Partitioning

To simulate realistic institutional heterogeneity, the training set was partitioned across five simulated clients using a Dirichlet distribution ($\alpha = 0.5$) over the class label, producing clients with substantially different fraud ratios and dataset sizes, summarized in Table 3.1.

Client	Total Rows	Fraud Cases	Fraud Rate
client_0	32,749	14	0.043%
client_1	628	111	17.675%
client_2	21,028	153	0.728%
client_3	14,144	4	0.028%
client_4	159,296	112	0.070%

Table 3.1: Non-IID client data distribution generated via Dirichlet partitioning ($\alpha = 0.5$)

This partition exhibits two independent axes of heterogeneity: label-distribution skew (client_1 has a fraud rate over 400 times higher than client_3) and dataset-size skew (client_4 holds approximately 253 times more data than client_1). Both dimensions are directly relevant to the federated learning and differential privacy analyses conducted in later chapters.

3.3 Technology Stack

Component	Tool / Library
Federated learning framework	Flower (flwr)
Model training	PyTorch
Differential privacy	Opacus

Component	Tool / Library
Simulation backend	Ray
Dataset	Credit Card Fraud Detection (Kaggle)
Evaluation and visualization	scikit-learn, matplotlib

Table 3.2: Technology stack used in this project

3.4 Hardware and Environment

All experiments were run locally on a single machine (Apple Silicon, macOS) using Flower's simulation mode, which emulates multiple concurrent federated clients as separate Ray actor processes on the same host. This is a standard and accepted approach for federated learning research where deployment across physically distributed institutions is not feasible for a student project; the simulation faithfully reproduces the algorithmic behaviour of FedAvg, FedProx, and per-client differential privacy, though it does not model network latency or communication failure between real distributed nodes.

Chapter 4: System Design and Methodology

4.1 Overall Architecture

The system follows the standard federated learning workflow. The central server initializes a global model and, for a fixed number of communication rounds, distributes the current global model to all participating clients; each client trains locally on its own partition for a small number of epochs and returns updated weights (and, in the differential privacy configuration, a per-sample-gradient-noised update); the server aggregates the received updates according to the active strategy (FedAvg or FedProx) and forms the next round's global model. This cycle is illustrated conceptually as: Global Model to Clients to Local Training to Weight Aggregation to Updated Global Model, repeated for fifteen rounds in the baseline experiments.

4.2 Model Architecture

The classification model (FraudNet) is a lightweight feedforward neural network consisting of an input layer (30 features), a hidden layer of 64 units with ReLU activation and dropout ($p = 0.3$), a second hidden layer of 32 units with ReLU activation, and a single-unit output layer producing a fraud logit. The model is kept intentionally small, since the project's central contribution concerns the federated and privacy-preserving training mechanism rather than model capacity.

4.3 Class Imbalance Handling

Given the dataset's extreme class imbalance (0.173% fraud), the binary cross-entropy loss is weighted using a positive-class weight computed from the square root of the negative-to-positive sample ratio. An unmodified ratio was found empirically to produce a degenerate model (precision = 0.052) that over-predicted fraud; the damped square-root weighting recovered a balanced operating point (precision = 0.582, recall = 0.867 on the centralized baseline).

4.4 FedAvg and FedProx Aggregation

FedAvg aggregates client updates via a weighted average proportional to each client's local dataset size. FedProx additionally introduces a proximal term, $(\mu/2)$ times the squared L2 distance between local and global model parameters, added to each client's local loss during training, constraining local updates from drifting too far from the global model. Two proximal coefficients were evaluated: $\mu = 0.1$ and $\mu = 0.01$, to assess sensitivity to this hyperparameter under the project's severe non-IID partition.

4.5 Differential Privacy Design

Differential privacy is applied at the client level using Opacus's PrivacyEngine, which wraps the local model, optimizer, and data loader to perform per-sample gradient clipping (max norm = 1.0) and Gaussian noise injection calibrated by a noise multiplier. Two design iterations were implemented over the course of the project:

- **Fixed global noise:** a single noise multiplier applied identically to every client, regardless of dataset size.
- **Adaptive per-client noise:** the noise multiplier is scaled per client according to (a) dataset size alone, and (b) both dataset size and number of local training steps, to compensate for the disproportionate composition effects observed under the fixed-noise design (detailed in Chapter 6).

Correct cumulative epsilon accounting across communication rounds required persisting each client's privacy accountant history to disk between rounds, since Flower's client model is stateless by default and a fresh PrivacyEngine is otherwise instantiated on every call to a client's local training method, silently resetting the reported epsilon to a single round's value rather than the true cumulative value.

4.6 Membership Inference Attack Design

To empirically validate the privacy properties of the trained models, a loss-threshold membership inference attack was implemented. For a trained model, per-sample binary cross-entropy loss is computed on an equal-sized sample of training-set ('member') and held-out test-set ('non-member') examples. A threshold at the median combined loss is used to classify each sample as a predicted member or non-member, and attack accuracy is reported relative to the 0.5 baseline expected under no information leakage.

Chapter 5: Implementation

5.1 Project Structure

The codebase is organized into distinct modules: data preparation (loading, preprocessing, and non-IID client partitioning), model definition and centralized baseline training, federated learning components (Flower client and server implementations, FedAvg and FedProx strategies), differential privacy utilities, membership inference attack scripts, and experiment orchestration and plotting scripts. Version control was maintained throughout development using Git, with all work pushed to a public repository (github.com/nitishpatil18/fl-fraud-detection).

5.2 Engineering Challenges

Three significant implementation issues were identified and resolved during development, each yielding an insight relevant to the project's findings:

5.2.1 Cross-Round Privacy Accounting

In the initial differential privacy implementation, a fresh Opacus PrivacyEngine was instantiated within each client's local training call, in accordance with Flower's stateless client design. This caused the reported epsilon to reflect only a single communication round's privacy loss rather than the true cumulative loss across all rounds in which a client participated, an error that would have substantially understated the actual privacy cost of the system. This was resolved by explicitly persisting each client's privacy accountant history to a per-client JSON file between rounds and reloading it at the start of each subsequent round's training call.

5.2.2 Privacy Accountant Selection

Following the cross-round accounting fix, simulations using Opacus's default Privacy loss Random Variable (PRV) accountant were observed to hang indefinitely for clients with large sample counts and many cumulative training steps. Diagnostic testing isolated the cause to expensive and numerically unstable FFT-based computation within the PRV accountant's epsilon calculation at high composition values, evidenced by floating-point overflow warnings during the computation. Switching to the Renyi Differential Privacy (RDP) accountant, which computes epsilon using a closed-form numerical search rather than FFT-based convolution, resolved the hang and reduced per-client epsilon computation time from over ten minutes to under two seconds, at the cost of a looser (larger) epsilon bound than PRV would provide.

5.2.3 Adaptive Noise Scaling

To address the disproportionate privacy loss observed for the project's smallest client (client_1, 628 rows), a noise-scaling function was implemented that adjusts each client's noise multiplier inversely to its dataset size. A subsequent iteration extended this to also account for the number of local training steps per client, since RDP composition scales with step count independently of dataset size. Both variants were empirically evaluated; the results and their implications are presented in Chapter 6.

5.3 Reproducibility

All experiments use a fixed random seed (42) for data splitting and client partitioning, and all training scripts accept configuration via environment variables (noise multiplier, number of rounds, batch size, local epochs), allowing each reported configuration to be reproduced exactly from the project's public repository.

Chapter 6: Results and Discussion

6.1 Baseline and Federated Learning Comparison

Variant	Precision	Recall	F1-Score
Centralized (baseline)	0.582	0.867	0.697
FedAvg	0.737	0.880	0.798

Variant	Precision	Recall	F1-Score
FedProx ($\mu = 0.01$)	0.646	0.896	0.744
FedAvg + DP (noise = 0.1)	0.762	0.739	0.748
FedAvg + DP (noise = 0.2)	0.706	0.763	0.729

Table 6.1: Final-round precision, recall, and F1-score across all evaluated variants

FedAvg ($F1 = 0.798$) exceeds the centralized baseline ($F1 = 0.697$) despite training on a non-IID partition, an outcome attributable to the aggregation process exposing the global model to varied fraud patterns across communication rounds. FedProx with $\mu = 0.01$ achieves a higher recall (0.896) at the cost of precision (0.646) relative to FedAvg, representing a different, and situationally preferable, point on the precision-recall tradeoff for a fraud-detection use case where missed fraud is typically costlier than a false alarm. Sensitivity testing found $\mu = 0.1$ to be too aggressive, degrading F1 stability (observed range 0.57 to 0.74 across rounds) relative to $\mu = 0.01$ (observed range 0.66 to 0.74).

6.2 Differential Privacy: The Composition Problem

Under the corrected, cumulative epsilon accounting described in Chapter 5, no combination of noise multiplier (0.15 to 0.5) and round count (3 to 10) tested produced both a meaningful privacy budget (epsilon less than approximately 10, a commonly cited threshold for a strong guarantee) and non-zero model utility simultaneously. Lower noise levels allowed the model to converge but produced epsilon values in the hundreds; higher noise levels kept epsilon low but prevented the model from learning the fraud pattern at all ($F1 = 0.0$). This represents a fundamental, not merely a tuning, limitation of the naive fixed-noise design at this dataset scale and class imbalance.

6.3 Membership Inference Attack Results

Model	Attack Accuracy	Attack AUC
Centralized baseline	0.494	0.500
FedAvg (no DP)	0.499	0.493

Table 6.2: Loss-threshold membership inference attack results (0.5 = no leakage detected)

Both the centralized and FedAvg-trained models show attack accuracy indistinguishable from random guessing (0.5), indicating no measurable membership leakage under this attack. This is attributed to the model's small capacity, dropout regularization, and limited training duration, which together limit memorization of individual training samples. This finding is notable when read alongside Section 6.2: it suggests that in this specific setting, the actual privacy risk being protected against by differential privacy may be small relative to the utility cost the DP noise imposes, at least as measured by this attack method. A more sophisticated attack (e.g., a shadow-model based attack) may reveal leakage this simpler method does not detect, and is noted as a direction for future work.

6.4 Adaptive Per-Client Noise Scaling

Configuration	Client 1 epsilon (628 rows)	Client 4 epsilon (159,296 rows)	F1
Fixed noise (no scaling)	98 - 320+	485,320	0.0 - 0.75 (unstable)
Size-only adaptive scaling	4.99	485,320	0.74
Size + step adaptive scaling	157.2	506.5	not separately measured

Table 6.3: Epsilon by client under three noise-scaling configurations, 10 rounds

Scaling noise inversely to dataset size successfully reduced the smallest client's epsilon from the hundreds to 4.99, a genuinely strong privacy guarantee, while maintaining $F1 = 0.74$. However, this scaling left the largest client's epsilon unchanged and effectively meaningless (485,320), since size-based scaling did not account for that client's proportionally much larger number of local training steps. Extending the scaling formula to also account for step count reduced the largest client's epsilon by approximately three orders of magnitude (to 506.5), but in doing so increased the smallest client's epsilon back up to 157.2, since that client's very low step count (4 steps per round) caused the step-based factor to counteract the size-based protection it previously received.

This result demonstrates that dataset size and step count are independent axes of heterogeneity that pull a single global noise-scaling formula in opposing directions: no formula tested could simultaneously provide strong protection to a client with very few, very informative steps and a client with very many, less individually informative steps. This motivates per-client target-epsilon calibration, where noise is solved backward from a desired epsilon per client, as a more principled direction than any single global scaling formula.

6.5 Summary of Findings

- FedAvg matches or exceeds centralized training performance under this non-IID partition ($F1$ 0.798 vs 0.697).
- FedProx's proximal term is μ -sensitive: an under-tuned or over-tuned μ can degrade rather than improve stability relative to plain FedAvg.
- Naive fixed-noise differential privacy exhibits a fundamental, not merely a tuning, incompatibility between meaningful privacy budgets and usable utility at this dataset scale.
- Empirical membership inference testing found no measurable leakage in either the centralized or FedAvg model, suggesting the models' actual privacy risk profile may be lower than the theoretical worst-case DP analysis assumes.
- Adaptive per-client noise scaling based on dataset size alone can achieve a strong, meaningful privacy guarantee for a small client without sacrificing model utility.
- Extending scaling to include step count improves protection for large clients but at the cost of the small client's protection, showing that no single global formula optimizes privacy for all clients simultaneously under heterogeneous non-IID data.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

This project implemented and empirically evaluated a privacy-preserving federated learning system for fraud detection, progressing beyond a standard FedAvg/FedProx comparison to identify and correctly resolve a genuine cross-round privacy accounting error, diagnose and fix a numerical stability issue in differential privacy computation, empirically validate privacy claims via membership inference testing, and investigate the limits of adaptive per-client noise calibration under heterogeneous client data. The central technical finding is that, at this dataset scale, differential privacy in federated learning cannot be treated as a single tunable hyperparameter applied uniformly across clients; heterogeneity in both dataset size and training step count must be addressed jointly, and no single closed-form global scaling formula tested in this project achieves this. The project also demonstrates, through federated learning results alone, that FedAvg is a practical, effective approach for privacy-conscious collaborative fraud detection even without differential privacy, given that empirical membership inference testing found no measurable leakage in the plain federated model.

7.2 Future Scope

- Implement per-client target-epsilon calibration, solving for each client's required noise multiplier given a fixed desired epsilon, rather than scaling a shared base noise value.
- Evaluate more sophisticated membership inference attacks (e.g., shadow-model based attacks) to more rigorously test the models' actual privacy leakage.
- Extend the federated simulation to a larger number of simulated clients and evaluate scalability of the adaptive noise scaling approach.
- Deploy the system across physically distributed nodes to evaluate the effect of real network latency and client dropout on convergence, beyond what the single-machine simulation can capture.
- Explore secure aggregation protocols to provide protection against a potentially adversarial central server, complementing the client-side differential privacy already implemented.

References

- [1] McMahan, H. B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS.
- [2] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. (2020). Federated Optimization in Heterogeneous Networks (FedProx). MLSys.
- [3] Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., and Zhang, L. (2016). Deep Learning with Differential Privacy. ACM CCS.
- [4] Dwork, C., and Roth, A. (2014). The Algorithmic Foundations of Differential Privacy. Foundations and Trends in Theoretical Computer Science.
- [5] Shokri, R., Stronati, M., Song, C., and Shmatikov, V. (2017). Membership Inference Attacks Against Machine Learning Models. IEEE S&P.
- [6] Beutel, D. J., et al. (2020). Flower: A Friendly Federated Learning Research Framework.
- [7] Yousefpour, A., et al. (2021). Opacus: User-Friendly Differential Privacy Library in PyTorch.
- [8] Kaggle. Credit Card Fraud Detection Dataset (mlg-ulb/creditcardfraud).

Note: the citations above are provided from general knowledge without live database access and should be independently verified (author names, venue, year, and page numbers) before inclusion in the final submitted report.